Tenfold: Celebrating 10 Years of Ink & Switch

Project Cambria

Translate your data with lenses

Ink & Switch
Geoffrey Litt
Peter van Hardenberg
Orion Henry
October 2020

Distributed software is everywhere. We consume or provide APIs, connect to databases, and load files written by others. As these systems evolve, we need to change data schemas while maintaining compatibility between different versions and components. But we don't have powerful tools for handling this schema evolution, so we usually resort to ad hoc solutions, like conditionals peppered throughout the code and complex multi-step migration processes.

We propose a principled replacement for these messy solutions: an isolated software layer that translates data between schemas on demand. This layer allows developers to maintain strong compatibility with many schema versions without complicating the main codebase. Translation logic is defined by composing *bidirectional lenses*, a kind of data transformation that can run both forward and backward.

We have implemented these ideas in Cambria, a TypeScript lens library, and used it to build an experimental issue tracker. The issue tracker has full support for real-time collaboration between all of its versions, and is demonstrated throughout this piece.

We believe tools like Cambria could revolutionize the way developers handle schema change. The benefits apply in many familiar contexts, but take on particular importance in decentralized software, where no single authority controls the data format. Ultimately, we hope that better schema evolution tools will lead to a "Cambrian Era" for software, where people can customize their software more freely while preserving the ability to collaborate with one another.

Original document

```
{
    "id": 1,
    "state": "open",
    "title": "Found a bug",
    "body": "I'm having a problem w
    "labels": [
      {
        "id": 208045946,
        "node_id": "MDU6TGFiZWwyMDg
        "url": "https://api.github.
        "name": "bug",
        "description": "Something i
        "color": "f29513",
        "default": true
      }
    ]
  }
```

→

Lens

```
# Rename body to description
- rename:
    source: body
    destination: description

# Convert boolean state to string status
- rename:
    source: state
    destination: status

- convert:
    name: status
    mapping:
      - { open: todo, closed: done }
      - { todo: open, doing: open,
          done: closed}

# Extract name of first label as category
- head: { name: labels }

- in:
    name: labels
    lens:
      - rename:
          source: name
          destination: category

- hoist:
    name: category
    host: labels

- remove: { name: labels }
```

→

Evolved docu

```
{
  "status": "to
  "description"
  "id": 1,
  "title": "Fou
  "category": "
}
```

A lens that translates between two issue-tracker formats. This lens is live, running code, as are all the examples in this document.

We welcome your feedback: @inkandswitch or hello@inkandswitch.com.

Contents

The challenge of schema change over time

Why is change so hard? If we could update our software in lockstep everywhere, life would be easy. Database columns would be renamed simultaneously in clients and servers. Every API client would jump forward at once. Peers would agree to new protocols without dispute.

Alas, because we can't actually change all our systems at once, our changes must often preserve both:

Backward compatibility, making new code compatible with existing data. **Forward compatibility**, making existing code compatible with new data.

Backward compatibility is straightforward. It's the ability to open old documents in new versions of the program. Forward compatibility, the ability to open documents in formats invented in the future, is rarer. We can see forward compatibility in web browsers, which are written so the features added to HTML won't break the ability to render new sites in old browsers.

> As any web developer already knows, writing forward-compatible HTML is more art than science. The Mozilla Developer Network maintains this guide with advice on the subject.

The need to maintain backward and forward compatibility appears in a wide variety of distributed systems, both centralized and decentralized. Let's look more closely at several of these systems and the solutions they employ.

Stripe API versioning

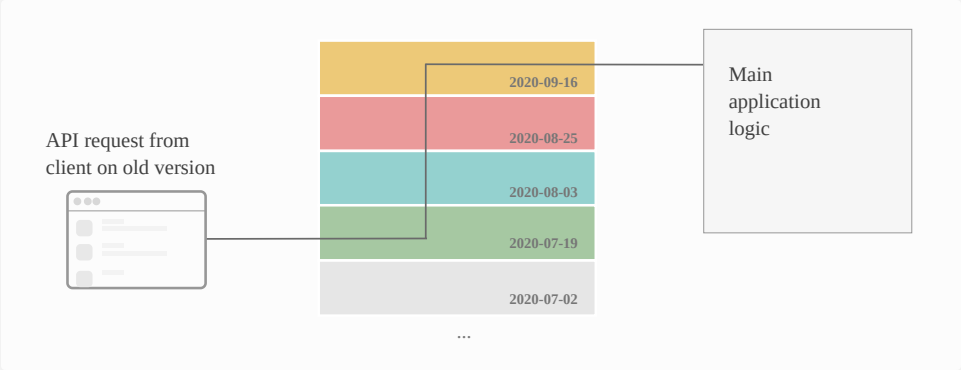
Public web APIs can face compatibility challenges when trying to maintain backward compatibility with older clients. An organization like Stripe must balance their desire to change their API against their customers' reluctance to change something that works for them. The result is strong pressure to preserve backward compatibility over time, often across many versions.

Many API developers take an ad hoc approach to this problem. Developers rely on tribal knowledge to inform them which operations are safe—for example, they intuit that they can respond with additional data, trusting existing clients to ignore it, but *not* require

additional data in requests, because existing clients won't know to send it. Developers also often resort to *shotgun parsing*: scattering data checks and fallback values in various places throughout the system's main logic. This can often lead not just to bugs, but also security vulnerabilities.

The term *shotgun parsing* was introduced by Bratus & Patterson, and describes the habit of scattering parser-like behaviour throughout an application's code. In addition to complicating programs, inconsistencies in handling of data can result in security vulnerabilities, as described in the linked paper.

Stripe has developed an elegant approach to this problem. They have built a middleware system into their API server that intercepts incoming and outgoing communication, and translates it between the current version of the system and the client's requested version. As a result, developers at Stripe don't need to concern themselves with the idiosyncrasies of old API requests most of the time, because their middleware ensures requests will be translated into the current version. When they want to change the API's format, they can add a new translation to the "stack" in their middleware, and incoming requests will be translated to the current version.



Stripe uses an encapsulated middleware to translate API requests and responses between the current version and older versions still used by clients.

Our work is inspired by the encapsulation provided by this system, but Stripe's implementation has some limitations. Developers writing migration rules must implement translations by hand and write tests to ensure they are correct. And because Stripe's system uses dates to order its migrations, it is limited to a single linear migration path.

Kafka and message formats

Large organizations often embrace an event streaming architecture, which allows them to scale and decouple data entering their system from the processes that consume it. Apache Kafka is a scalable, persistent queue system often used for this purpose. Data schemas are paramount in these environments, because malformed events sent through the system can crash downstream subscribers. Thus, Kafka's streams can require *both backward and forward compatibility*. Old messages must be readable by new consumers; new messages must be readable by old consumers.

To help manage this complexity, the Confluent Platform for Kafka provides a Schema Registry tool. This tool defines rules that help developers maintain schema compatibility —for example, if a schema needs to be backward compatible, the Schema Registry only allows adding optional fields, so that new code will never rely on the presence of a newly added field.

Compatibility Type	Changes Allowed	Upgrade First
Backward	▪ Delete fields ▪ Add optional fields	Consumers
Forward	▪ Add fields ▪ Delete optional fields	Producers
Full	▪ Add optional fields ▪ Delete optional fields	Any order

An excerpt from Confluent's list of rules for preserving compatibility in Avro. Developers can pick a desired compatibility level, which then limits the changes they can make to a schema.

The Schema Registry can help prevent a team from deploying incompatible schemas, but it does so by limiting the changes that can be made to schemas. Developers who want to preserve compatibility cannot add new required fields or rename existing fields. Furthermore, these rules limit the guarantees provided by the schema; a record full of optional fields is hard to use, and code that processes it may resort to shotgun parsing, testing for the presence of each field before using it.

Mastodon protocol evolution

Decentralized systems introduce even more challenges. Historically, systems like BitTorrent, IRC, and email have been governed by a centralized *protocol*, despite decentralized implementations. Maintaining a strict, well-defined protocol allows each instance of the software to know what to expect and produce from its peers.

Mastodon is a federated Twitter-like social network where anyone can run their own server and decide which users and messages they'd like to include. Each Mastodon server is encouraged to develop its own community standards and culture that govern what kind of content is welcome.

In the social network Mastodon, servers are all independent, and exchange data using the open-ended ActivityPub standard. ActivityPub provides a shared format and defines many important elements of the distributed system. It specifies how "followers" and "likes" should operate, but new features can be difficult to add because of coordination challenges.

Each local server, of course, can do as it wants. For example, one could provide a local way of describing and hiding sensitive content. But with each system implementing these ideas locally, there is no easy way to disseminate these improvements globally. If two servers handle sensitive content in different ways, must each support both formats forever? To make matters worse, innovation tends to occur at the edges. Large servers are slow to adopt new features, and with many implementations out there, there's no clear path for administrators to choose which ones to adopt and when.

The more successful Mastodon (or ActivityPub) becomes, the more implementations and servers there are. The more implementations and servers, the harder it is to change

the protocol. The IRC network has struggled with these problems for many years, an issue so serious that one of the key features of the latest IRC protocol, IRCV3, is just the ability to make future changes! Perhaps if IRC had been more able to respond to the changing landscape, it would have been more able to compete with new services.

IRC, or Internet Relay Chat, is an early internet protocol used for real-time chat, and the progenitor of systems like Slack. In 2003, IRC claimed 10,000,000 simultaneous users online. By 2020, it had dropped to only 371,000.

10x harder: decentralized data schemas

At Ink & Switch, we've been exploring a flavor of decentralized software we call local-first. Because local-first software can run entirely on users' computers and yet support real-time collaboration, we've run into all kinds of problems with upgrading to new versions of code. Unmanaged interactions between different versions of programs have led to bizarre behaviour, such as clients competing to remove and restore relocated data, or invalid states in rendering code that result from replaying document histories created by an earlier version... or a later one.

A **distributed** system is any system with more than one computer coordinating.
A **decentralized** system has no central authority. BitTorrent is one, but so is email.
A **federated** system is a two-tier decentralized system where users connect to the server of their choice and the servers coordinate. BitTorrent is not federated, but email is.

Our initial solutions were ad hoc. Like everyone, we relied on shotgun parsing, doing things like inserting conditional type checks where we saw missing arrays. We'd hack one-off migration code into the project and remove it later when it "felt safe". The further along we got, with more developers working on the software, the worse things became.

We would frequently end up writing code like this snippet, which needs to handle two cases: the case where `doc` already has a `tags` array, and the case where it doesn't have one yet because older versions of the application didn't populate that property in the document.

```
if (doc.tags && Array.isArray(doc.tags)) {
  doc.tags.push(myNewTag);
} else {
  // Handle old docs which don't have a tags array
  doc.tags = [myNewTag];
}
```

This kind of code would often be added after the fact. New documents would be correctly initialized during creation, but old documents would be missing fields and only expose problems by throwing errors at runtime.

Worse yet were problems caused by renaming fields. Consider the following example where we rename `authors` to `contributors`:

```
if (doc.authors) {
  doc.contributors = doc.authors;
  delete doc.authors;
}
```

Is the bug apparent? This code will correctly migrate the field to its new name, but only if no old copies of the software are still working on this document! Old versions of the program accessing this document are liable to reintroduce the `authors` field, resulting in loss of data.

Of course, there are better ways to implement this migration, but we can see how subtle problems can emerge even if single-user tests behave correctly. These kinds of sequencing problems are rare in centralized systems, but common for decentralized systems.

Distributed schema evolution is a shared problem

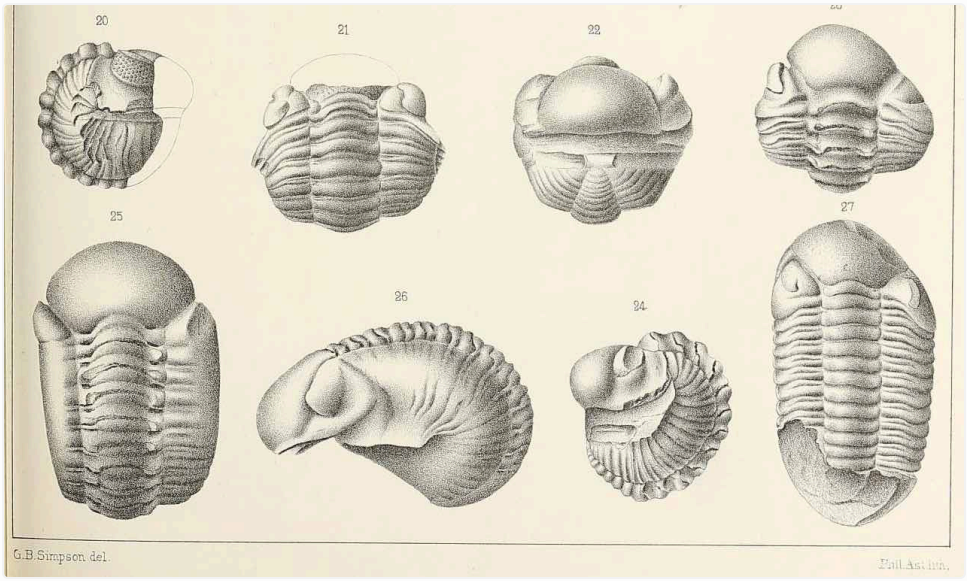
All of these systems must be designed to support multiple versions of data simultaneously. This is hard to do when the data is coming from API clients, it's hard to do when it's stored in an immutable Kafka queue, and it's especially hard to do when there are a whole gaggle of decentralized clients all evolving in arbitrary order.

Getting this right is no academic concern. Failure to correctly send and receive data in a distributed system can cause outages. Existing clients may lose the ability to communicate with a server. Network partitions can occur if only some nodes upgrade. Inconsistent implementations can lead to security vulnerabilities.

There is a deep sense in which these are all the same problem. Data schemas can only change in so many ways. Evolving schemas may add or remove fields, rename or relocate them, change their cardinality, or convert their type from one form to another. To manage this complexity, programs must check fields for existence, fill in defaults, and migrate old data into new shapes.

Instead of managing these problems in an ad hoc way, we can build on the insights of the Stripe and Confluent teams and provide a satisfying framework that solves them consistently and reusably across all these problem domains. We can make it radically easier to support many versions of data all at once.

Introducing a schema evolution tool: Cambria



Phacopida ("lens-face") is an order of trilobite that originated from the Late Cambrian Period. Photo courtesy of Biodiversity Heritage Library.

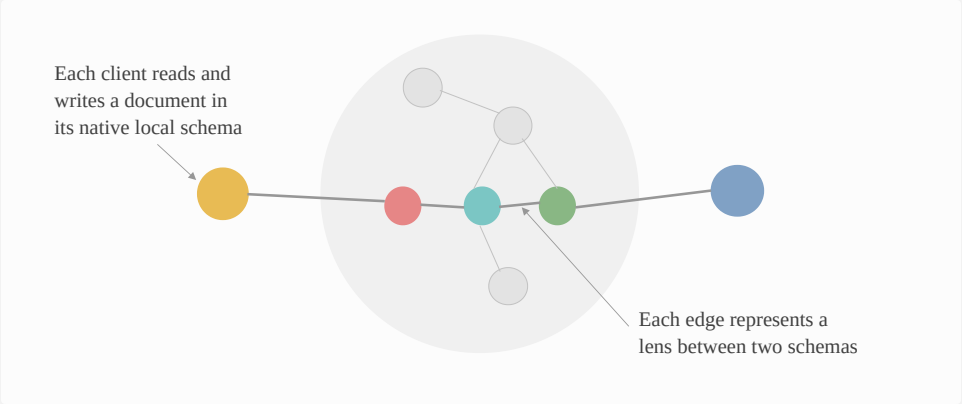
We have developed a schema evolution library called Cambria that supports real-time collaboration between different versions of an application with different schemas. It works by giving developers an ergonomic way to define bidirectional translation functions called *lenses*.

When a developer defines a single lens, it can translate data in both directions, eliminating the need to specify separate forward and backward translations. We have built on extensive academic research in this area, most directly "Edit Lenses" by Hofmann, Pierce, and Wagner. Our goal was to integrate these techniques into a practical toolkit that can be used to support compatibility in real applications.

Some readers may have seen lenses used as "functional getters and setters," as in the Haskell lens library. Here, we use them for synchronizing related data structures, which was the original purpose of lenses as envisioned by Foster et al. For more details on bidirectionality in Cambria and how it relates to prior work on lenses, see Appendix I.

Cambria is a lightweight library, designed to be integrated into JavaScript and TypeScript environments. It consists of a small set of functions that take data in one version and return it in another version. It works on JSON data from any source: another client over the network, a file, or a database. Cambria does compile-time validation to ensure that lenses are valid and consistent, and produces TypeScript types and JSON Schema definitions so that developers can reason about the shape of the data.

Over time, a project using Cambria will accumulate many lenses, each describing the relationship between two versions. Migrations between distant versions are created by composing lenses into a graph where each node is a schema, and each edge is a lens. To translate data between two schemas, Cambria sends it through the shortest available path in the lens graph. These lenses must be kept in a place where even old versions of the program can retrieve them, such as in a database, at a well-known URL, or else as part of the document itself.



Cambria maintains a graph of data schemas, connected by bidirectional lenses that translate data between them.

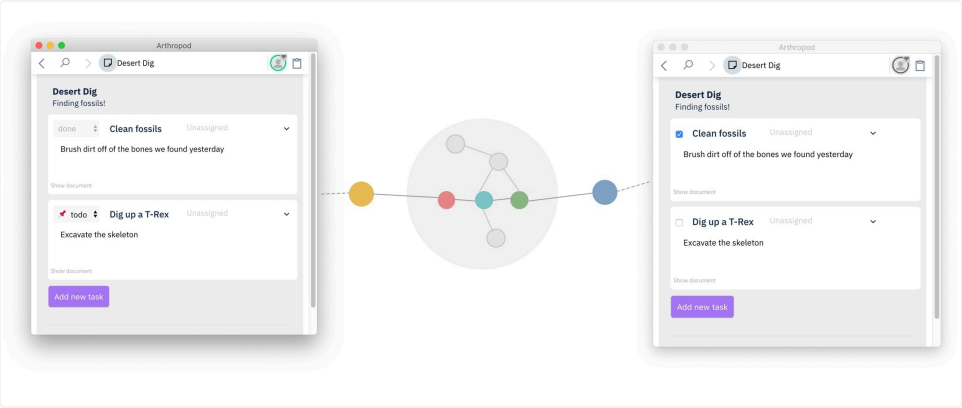
To understand what this all means in practice, let's see Cambria in action on a real project.

Working with Cambria

To develop Cambria in a realistic context, we built an issue tracker application. It's a local-first program that runs on each user's computer but also supports real-time peer-to-peer collaboration over the internet. This combination immediately introduces compatibility challenges. How can users collaborate if they are running different versions of the client?

While we chose to work in the context of a local-first application, many of the ideas in this demo are applicable to other contexts like centralized web APIs, event streams, and databases as discussed further below.

Our goal was to use Cambria to achieve full compatibility. Users should be able to collaborate on a document regardless of the software version they're running. New clients should be able to open documents created by old clients, and vice versa.



Issue tracker clients can collaborate in real time, even while running different versions. Each client interacts with a local document in its native schema; Cambria's translation layer ensures compatibility across them.

To gain experience with real-world schema evolution, we started using our issue tracker with a minimal feature set, and added features as we needed them, maintaining full compatibility the entire time. This helped us focus on genuine problems rather than imagined issues, but may have led us to ignore other types of schema change.

Although we did use our prototype issue tracker to help develop Cambria, we found it too unstable to be our only system of record during the project. The main problem was that we were changing the underlying storage format used by Cambria itself; this kind of problem seems unlikely to affect a project that was merely using Cambria and not developing it.

Let's look at some of the changes we made as our application evolved, and how lenses helped maintain compatibility throughout.

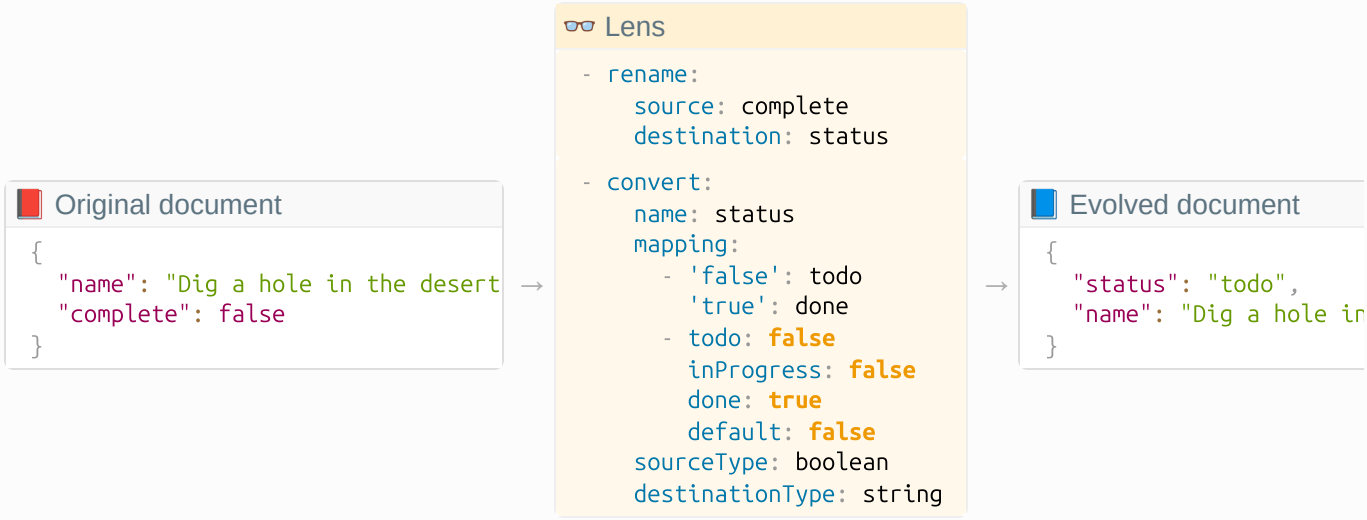
Lenses in action

Adding more issue statuses

Initially, our application tracked the status of each task with a boolean `complete` field. But as we used the software, we realized it was valuable to see whether an incomplete task had been started yet. So we decided to track status as a three-valued string type instead: `todo`, `in progress`, or `done`.

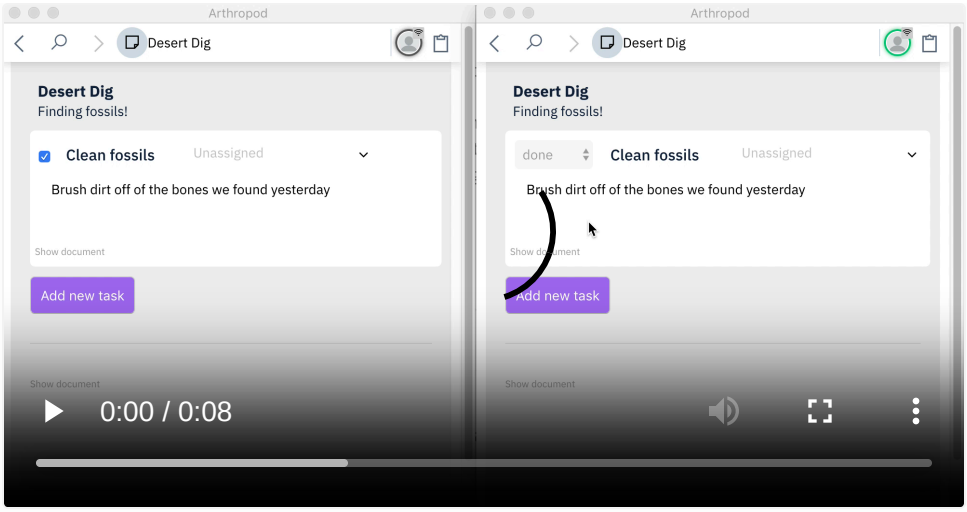
Using Cambria, we wrote a lens that expressed this conversion. In the diagram below, you can see the original JSON document on the left, the lens in the middle (using YAML syntax), and the evolved document on the right.

First, the lens renames the `complete` property to `status`. Then, it converts the value of the property from a boolean to a corresponding string. For example, here `complete: false` is mapped to `status: todo`.



A lens that converts a boolean complete value to a string status value.
Note that in the second step, we define mappings in both directions, so that a boolean can be converted to a string, and vice versa.

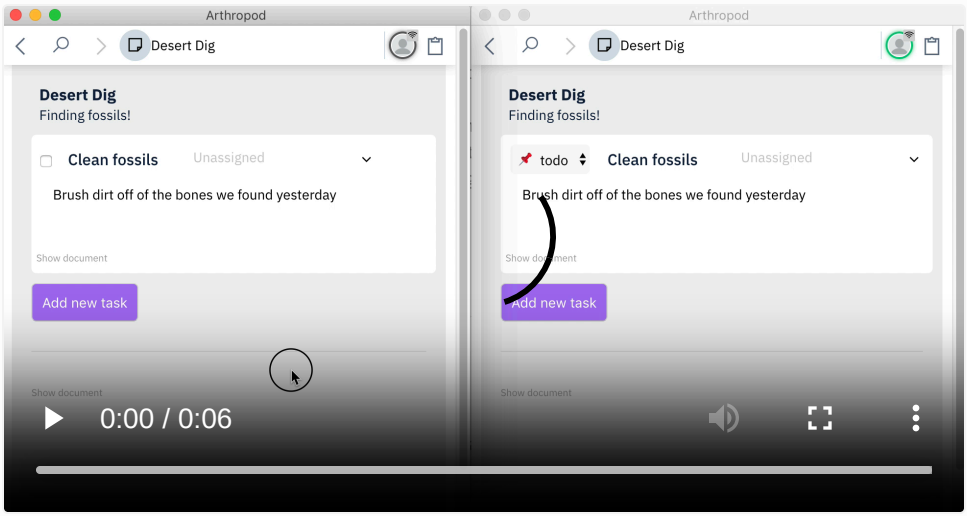
Let's see this lens in action in the actual application UI. Here we see two versions of the issue tracker running side by side, collaborating on the same task. On the left is the old version with a checkbox for the "Clean fossils" task; on the right is the new version with the dropdown. Changes from the new version propagate to the old version—watch how the checkbox on the left updates in real time as we make updates on the right side:



FORWARD COMPATIBILITY

When the dropdown status is changed in the new client, the old client shows the corresponding checkbox value. (Note how "todo" and "doing" both map to an unchecked checkbox.)

Changes from the old client translate to the new version, too:



BACKWARD COMPATIBILITY

When the checkbox is toggled in the old client, the new client shows the corresponding dropdown status. The new client is backward compatible with the old data format.

Each client edits the task using its native data structure, and Cambria translates those edits back and forth. In many other systems renaming a field between schema versions is a complicated process, but here it just requires writing a small lens.

Supporting multiple assignees

Let's consider another change. At first, our issue tracker supported assigning one person to each task. Then, later in the project, we started doing a lot of pair programming, so we wanted to be able to assign multiple people.

As in the previous example, perfect compatibility is impossible. The old client knows only how to display a single assignee. If someone uses the new client to assign multiple people to a task, there's no way to present that information in the old client.

However, we can still make a best effort to maintain compatibility. We can write a lens using the `wrap` operator, which translates a scalar value into an array. When the array contains a single element, both sides show the same information. When multiple elements are present in the array, the scalar value reflects the first element in the array.

There are many different ways to formulate a scalar-to-array conversion, each with subtly different properties. See Appendix III for more details.

Original document

```
{
  "name": "Dig up a T-Rex",
  "assignee": "Bob"
}
```

→

Lens

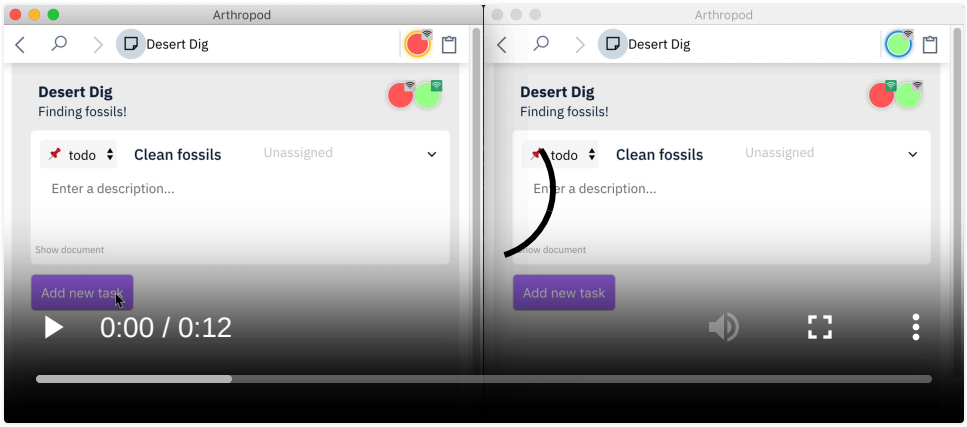
```
- rename:
    source: assignee
    destination: assignees
- wrap: { name: assignees }
```

→

Evolved document

```
{
  "assignees": [
    "Bob"
  ],
  "name": "Dig up a T-Rex"
}
```

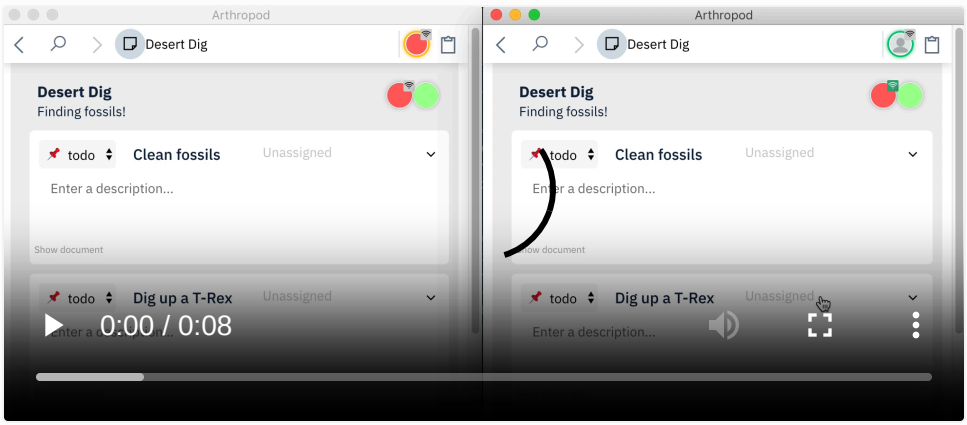
Let's see how that lens behaves in the application, once again with two versions side by side. The old version on the left supports a single assignee; the new version on the right supports multiple assignees. When the old version edits the single assignee for a task, both sides stay perfectly in sync:



A SINGLE ASSIGNEE

The two applications show the same information when only one person is assigned to a task.

When the new version assigns multiple people to a task, the old version does the best it can by displaying the first assignee in the list:



MULTIPLE ASSIGNEES

When multiple people are assigned to a task, the old version shows the first assignee in the list.

Depending on the context, less-than-perfect translations like these might be useful or confusing. If at some point the costs outweigh the benefits, the best option might be to require all collaborators to upgrade. Cambria doesn't force users to collaborate across versions with imperfect compatibility, but it provides the option of doing so.

Adding tags to issues

The above examples show relatively sophisticated evolutions, but Cambria can also help with managing more mundane changes like adding a new field.

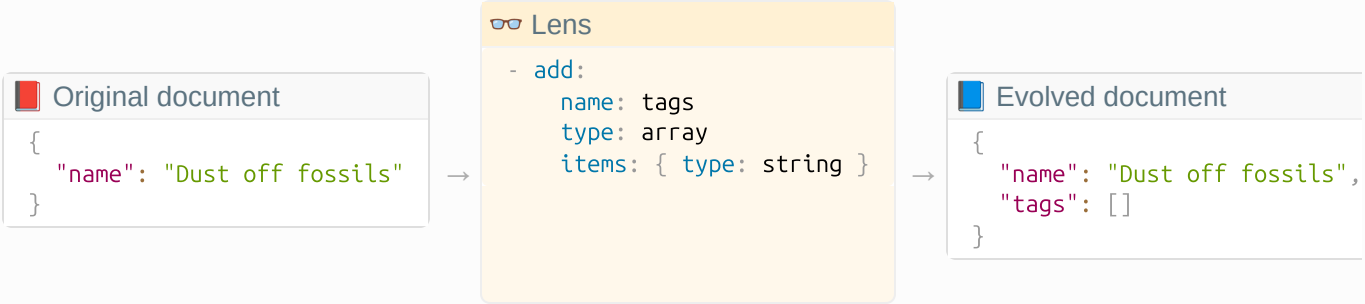
For example, at one point we decided to add an array of tags to each task, to help keep track of different streams of work. Even this small change causes backward compatibility challenges, since the new code needs to work with older documents where the tags array is nonexistent. Naive code like this will throw an exception if the `tags` property isn't present:

```
const processedTasks = task.tags.map(processTask);
// => Uncaught TypeError:
//      Cannot read property 'map' of undefined
```

Using ad hoc shotgun parsing, we could fill in a default value at the time the data is being used, but this has the downside of mixing data validation into the main program:

```
const processedTasks = (task.tags || []).map(processTask);
```

With Cambria, we can achieve a stronger separation of concerns by using a straightforward lens to add the new field.



The `add` lens operator populates an empty array as the default value. This means we can switch back to the "naive" code, but this time we can use it with confidence, since `task.tags` is guaranteed to always be an array.

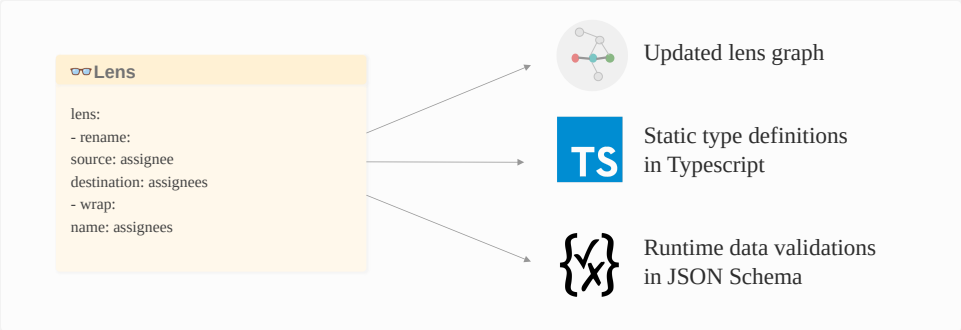
Developer workflow

We've now seen a few examples of how lenses can help conceptually with evolving data. But another important part of Cambria is the *developer workflow* for actually using these lenses ergonomically in an application. In this section, we'll show how developers interact with lenses, and how Cambria provides tools to make it easier to reason about data schemas while programming.

Lens once, use everywhere

A key design principle in Cambria is that there should only be a single source of truth describing the expected shape of data in a system, and how that shape has evolved over time. To change this schema, a developer only needs to write a single lens describing the desired change, and Cambria automatically produces three artifacts:

- **an updated graph of lenses**, which converts data between versions at runtime
- **updated TypeScript types**, to ensure that the program is compatible with the new schema
- **updated JSON Schema**, to validate incoming data at runtime



Producing all of these artifacts from one lens specification reduces duplication of effort and saves time, but most importantly it ensures that different components stay correctly in sync. For example, we won't forget to handle a data-compatibility edge case when editing TypeScript types, because both are handled by the same system.

Registering a lens

Using Cambria in practice consists of adding the library to an application, then writing lenses, and using some shell commands packaged with the library to build the artifacts described above.

Let's see this in action by showing how to add the "multiple assignees" lens discussed above.

To create a new lens, we first run a shell command that generates a time-stamped YAML configuration file in the `src/lenses` directory. We specify that we want to edit the `Task` data type, and we give a descriptive name to our lens: `addMultipleAssignees`.

This interface is designed to echo database schema management tools, in particular the Ruby on Rails command `rails generate migration`. Cambria is agnostic about package managers, but we happen to use yarn. As with other schema management tools, we use time-stamped files as an ersatz vector clock for ordering lenses committed separately by multiple developers and later merged together.

```
$ yarn lenses:add Task addMultipleAssignees
```

We edit the contents of the new file to contain our lens body:

```
schemaName: Task
lens:
  - rename:
    source: assignee
    destination: assignees
  - wrap: { name: assignees }
```

Then we run a final command to "build" the lens:

```
$ yarn lenses:build
```

This command registers the new lens into the application's graph of lenses by connecting it to the latest lens for the `Task` data type. In essence, we have added the new lens to the collection of transformations for `Task`, making the new logic available to use for data evolution at runtime.

Generating static types

The lens build process also produces TypeScript types. When the lens is registered, Cambria updates the TypeScript type for `Task`, renaming the field and changing its type from a nullable string to an array of strings:

```
  interface Task {
    name: string;
-   assignee: string | null;
+   assignees: string[];
  }
```

Having an updated type makes it trivial to identify all the places in the code that need to change to accommodate the new schema. For example, here TypeScript helpfully suggests that we update a property name from `assignee` to `assignees`:

```
294  const assignee = task.assignee
⊗ patch.ts 2 of 2 problems
Property 'assignee' does not exist on type 'Task'. Did you mean 'assignees'? ts(2551)
```

Generating runtime schemas

Our static type system helps during development, but does nothing for us at runtime. Malformed incoming data could still cause errors, so we need a way to validate data during program execution.

To help with this, Cambria generates runtime-checkable schemas using the JSON Schema standard. The lens build process produces an updated schema definition reflecting the changes made by our new lens:

```
  {
      "$schema": "http://json-schema.org/draft-07/schema",
      "title": "Task",
      "properties": {
        ...
-         "assignee": { "type": ["string", "null"] },
+         "assignees": {
+           "type": "array",
+           "items": { "type": "string" }
+         },
      ...
      }
  }
```

At runtime, we use this schema to validate incoming data. If the data doesn't pass initial validation, our program can fail early and provide a helpful error message rather than crashing deeper in the main program logic.

When all data comes from different versions of one codebase, malformed data is unlikely. But in other cases like a public web API, it's common to receive data that doesn't match the expected schema.

In this section, we've focused on the experience of using Cambria, both as a developer and as an end user of an application. For more details on the internal implementation, see Appendix I, which describes how Cambria works, and Appendix II, which describes its integration with the Automerge CRDT library.

Findings

By using Cambria over several months to build a real application, we hoped to gain some insight into the experience of using this kind of tool in practice. Here are some of our noteworthy observations.

Interoperability requires trading off between irreconcilable design goals.

Today, we generally tend to view compatibility as a binary: two systems are either technically compatible, or not. But with a flexible schema evolution system, compatibility becomes more of a human-centered design question: how far can two pieces of software diverge before it becomes impossible to usefully collaborate between them? Lens evolutions cannot magically make two incompatible pieces of software work perfectly together; the goal is merely to enable developers to maximize compatibility within realistic limits.

We found that when two applications use sufficiently different data representations, any interoperability design requires making tradeoffs between several desirable design goals:

- **consistency**: both sides see a meaningfully equivalent view of the world
- **conservation**: neither side operates on data they can't observe
- **predictability**: the local intent of every operation is preserved

Consider once again the example of two versions of our issue tracker: one with a single assignee per task, another with multiple assignees. When there are multiple people assigned to a task, the older app version only shows the first assignee. Let's say a user on the older version removes that single assignee. What should happen in the multiple-assignee version?

First, we'll demand consistency as non-negotiable. If both sides don't see some meaningfully related view of the same reality, we're not really working on the same data.

We could interpret the "delete" operation as requesting deletion of all the elements in the array, but now we've invalidated our **conservation** goal. The old version of the program has unassigned other people from the task without knowing they were assigned in the first place.

Another option is to just remove the first element of the array. But now, consistency demands that the old program should change the single assignee to a different value: the new first assignee. For a user of the old program, it's surprising to remove an assignee and see a different name appear in the box; this option violates the **predictability** goal.

> For a longer discussion of this issue which surveys a number of different options, see Appendix III.

This example demonstrates that there are limits to interoperability. Sometimes, there are no perfect options. But we view that fact as a challenge, not a dealbreaker. While schema evolution tools can't magically resolve certain fundamental differences, they can still maximize compatibility across applications, stretching the ability to collaborate as far as possible.

Of course, if a group of collaborators wants to achieve closer compatibility, they still have the usual option of all switching to the same software version. Schema evolution tools simply expand the possibility space further.

Combining types and migrations makes life easier.

On a purely subjective level, we were surprised as a team by how much we enjoyed having both TypeScript types and data translations produced from the same code. The confidence that we had updated every reference to old property names or types made development run more smoothly. Similarly, the ability to trust that the data we got off the disk or wire would be compatible with our running code—even as we changed the schema—made introducing new features easier. Overall, the developer experience *feels* akin to working with ActiveRecord Migrations. A small amount of upfront diligence pays big dividends.

Data schemas aren't linear, even in centralized software.

Even centralized software rarely has a linear version history. Developers routinely build features on branches, share them for review and testing, and then merge them. When these branches include schema changes, reviewers must decide whether to somehow migrate their local data to and from that branch, or else create a new database. Being able to move back and forth across branches and schemas without risk can make development more comfortable.

Data translations in decentralized systems should be performed on read, not on write.

Some of our early prototypes for storing documents that were compatible with multiple schemas performed data translations at write time. When writing a change to a document, a client would also translate the change into all the other related schemas that it knew about.

We eventually realized this was a flawed strategy. It struggled to handle new schemas getting added later on, after the write had already happened. We came up with workarounds, but they struggled to handle tricky situations like a write happening concurrently with a new schema being registered in the document. As a secondary concern, it was also difficult to make this design perform well, since too much translation work was happening eagerly on write.

So we switched to a simpler strategy: store a log of raw writes in the form of the writer schema, and translate between versions at read time. In this design, it's not a problem if more schemas are introduced after the write occurs, as long as the *reader* knows about the necessary lenses for translating between the writer schema and the reader schema. This design also performs better because translations only happen lazily when a reader actually needs the document in a certain schema.

This principle guided the design of our final integration between Cambria and the Automerge CRDT system. See Appendix II for more details.

Lenses require well-defined transformations.

Many apparently simple data transformations have surprising complexity. Imagine a lens that combines a `firstName` and `lastName` fields into a single `fullName`. This lens works reliably in one direction. All names already stored in the system could be combined into a single field, but there are many names which could not be reliably converted back to "first" and "last" names. The result is a so-called lens that can only run reliably in one direction. It might be practical to support transformations which do not succeed on all possible data and produces errors on some input. We have not yet explored these kinds of transformations, but we believe they would be useful.

As with many falsehoods that programmers believe, names have surprising complexity. Although Americans often assume users will have a single "first" and "last" name separated by a space character, this is by no means true around the world. Some last names ("van Hardenberg") include spaces, as do some first names ("Sher Minn"). Some people have a single name ("Madonna"). In Japan, family names are written first, then given names. For an in-depth treatment of how to correctly handle names, we recommend this enjoyably written W3C guide, but our brief recommendation is that you ask your users how they like to be addressed, and what to call them in a formal context.

Open questions and potential for further research

As a research group, we do not pretend to have delivered a fully formed, production-quality solution. Rather, we hope we've convinced you that an alternative approach to schema change is not only imaginable, but can be made practical.

Here are some of the open questions still remaining for this work, which we hope others will continue to explore with us.

Additional lenses

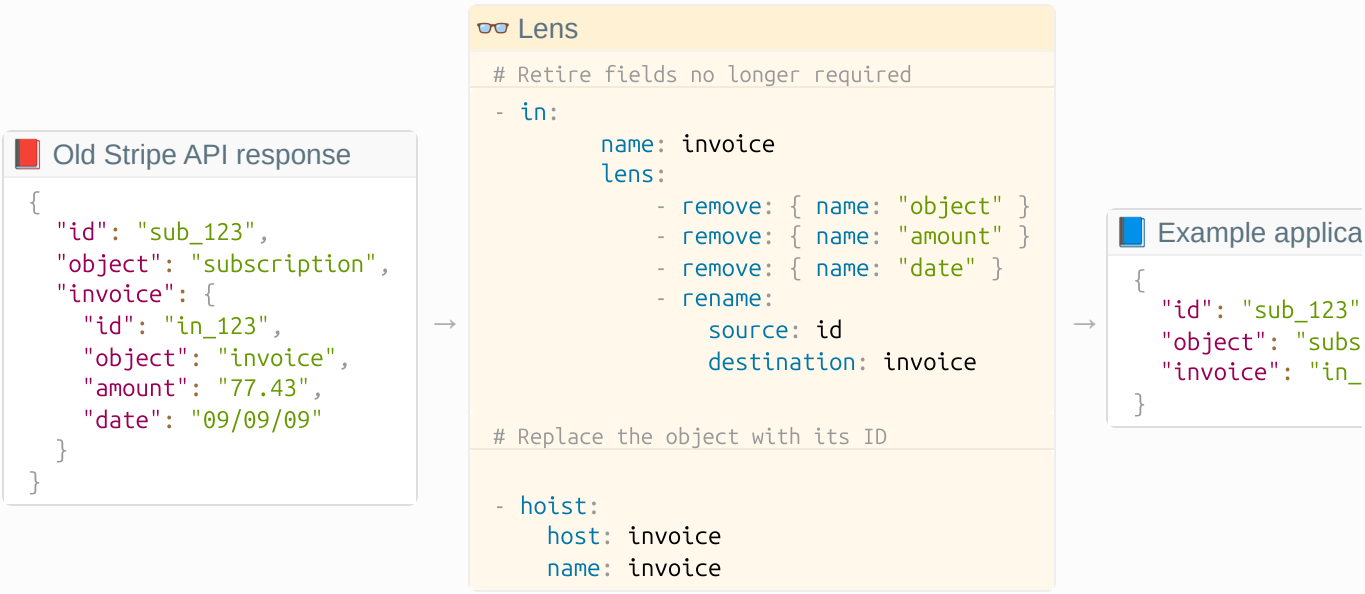
Throughout this essay we have demonstrated a number of the lenses we built to support our issue tracker prototype. These cover many of the kinds of data schema changes and conversions we've seen in real-world projects, but there are many other lenses yet unbuilt. Future work should explore lenses that deal with sorting, filtering, and merging or splitting arrays, as well as more complex value conversions such as date formats. Some of our existing lenses would benefit from more configuration options. For example, when converting between arrays and scalar values, we currently provide a `head` lens, but perhaps a `tail` or a `lastAdded` lens would be helpful for some applications.

In addition to a broader range of fully bidirectional lenses, transformations that operate on a limited set of data may also be useful. Some useful schema evolutions might only work successfully on a subset of data, or only in one direction. For example, a new license plate format might accept all existing data but also some new data that old programs could not successfully interpret. These would not, strictly, be lenses.

Augmenting data

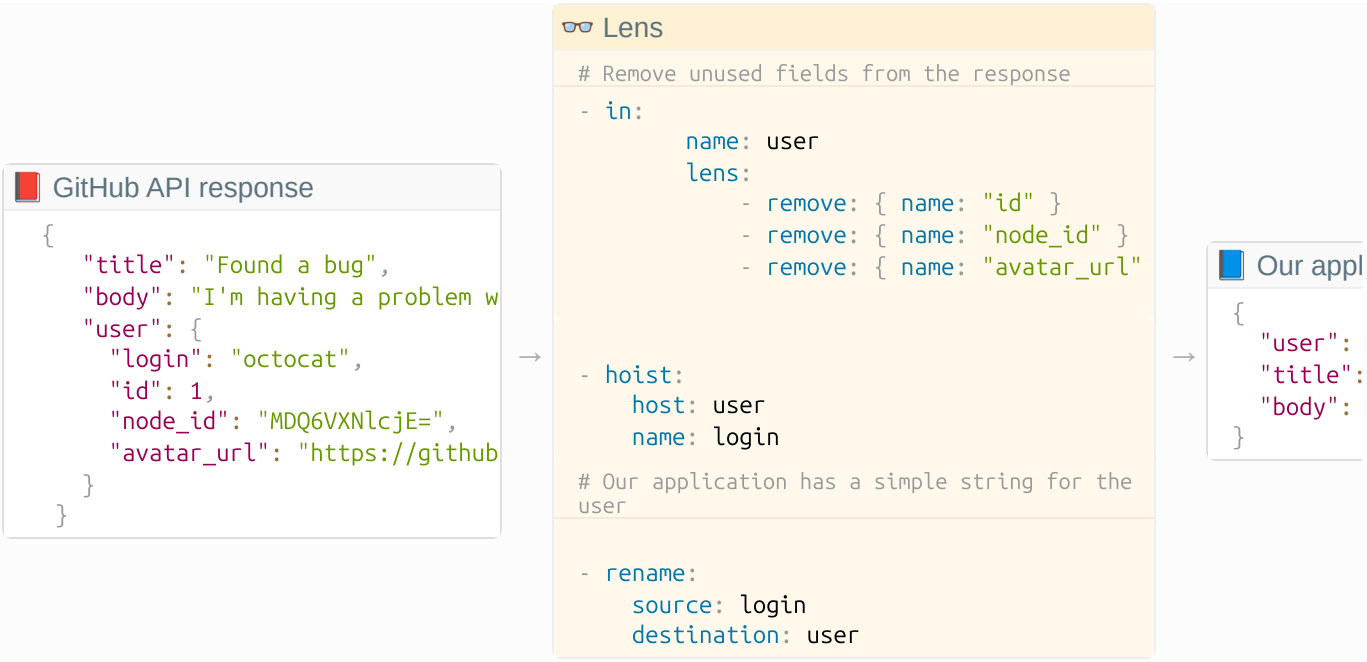
A change to a schema may require new data, or remove previously required data. Although Cambria has support for providing default data in the case of added or removed fields, it does not provide a mechanism to look up missing data, and so cannot support this kind of change today.

Consider this example (courtesy of Stripe's Brandur Leach), where an inline object (an "invoice") is moved out to just an ID.



What should the reverse of this lens be? Older clients will need real data, not stand-in values. To solve this problem, Cambria needs a way to express dependencies on other data sources, or perhaps some kind of callback to allow a lens author to fetch data from other sources. The interface for this problem is subtle, but important.

Similarly, we may need to interpret edits beyond simply applying a mechanical change. Consider this representative example of a GitHub API response that describes an issue in their issue tracker:



What does it mean when our application edits the `user` field? It probably suggests that the assignee for this issue has changed to another person, but when we run our lens backward, this change simply edits the `login` name for the GitHub user. Correctly interpreting this change means much more than simply editing a field. In fact, naively editing the `login` field would imply that we are editing the username of a GitHub user, which is almost certainly incorrect.

In the best case, we will need to look up and fill in the removed fields (like `id` and `avatar_url`) with new values that match the new user's account information. There are other, more problematic cases. The new user may not exist in GitHub's system, or have a different name, or we may not have permission to edit this field.

There is no simple solution to this problem. Some application logic outside of the lens system will be required.

Named lenses and recursive schemas

Some data schemas are recursive. For example, consider a task that has subtasks, which can in turn contain their own subtasks, and so on.

```
{
  "name": "Extract trilobite skeleton",
  "subtasks": [
    {
      "name": "Cut out of stone",
      "subtasks": [
        { "name": "pick away stone", "subtasks": [] },
        { "name": "remove with tweezers", "subtasks": [] }
      ]
    },
    {
      "name": "Clean skeleton",
      "subtasks": []
    }
  ]
}
```

Imagine we wanted to write a lens that renamed the `name` property within a task. It would need to recursively traverse this schema to an arbitrary depth, applying the transformation at each level.

Working across documents

Moving data between documents poses challenges. Splitting one document into many, or migrating data from one document to another, are obvious use cases here. Past systems like the XML-transforming language XSLT have struggled with combining and splitting documents.

Lens inception

Because Cambria's goal is to maintain backward compatibility with deployed lenses in the field, we'll need to consider how Cambria's own data might be versioned and lensed. In addition to requiring recursive lenses, we should consider compiling lenses to a portable bytecode like WebAssembly so that they can be executed by earlier versions of the library.

Performance

We have not yet measured Cambria's performance in any formal way. Lens pattern matching could be made faster, and computed lens paths could be optimized. The current version of Cambria-Automerge may apply an edit to many versions of the document. Our current design also implements document evolution as a special case of patch evolution. We atomize a document into a set of patches, run our lenses on each one, then reassemble the document at the end. This could likely be handled more efficiently by a direct implementation.

Lens repositories

Cambria-Automerge, used by our prototype issue tracker, stores lens data in documents themselves. This ensures forward compatibility because old code is able to load the lenses and translate the document into its native format. Instead of storing every lens in every document, it would be more efficient to share lenses directly.

Applying lenses to other domains

What we've found is that lenses are not so much a technology as an idea, and have the potential to be applied in a great many domains. The core ideas remain the same in all of them, to wit:

- mark data with versions
- relate versions through bidirectional lens descriptions
- apply chains of transformations to swap formats

Our prototype issue tracker applied lenses to the problem of collaborative document editing in local-first software, but we feel that lenses would be equally applicable in a wide variety of other domains.

Web APIs with slow-to-update clients (like Stripe)

Web APIs often have to support many different versions of their client libraries. This could be a great environment for a lens tool like Cambria. Requests are already generally tagged with versions either in the URL or in a header, so our first requirement is already satisfied. Lenses could layered into API code as middleware which translates incoming data from the sender's format to the locally required one, just like the Stripe example. As a preliminary exploration, we have built cambria-express, a bare-bones integration of Cambria with the Node Express web server that illustrates the basic idea.

> A more ambitious approach might be to model an entire API, endpoints, requests, and their responses, as one large type. We have not explored this approach in depth.

Document databases

Document databases like Mongo, CouchDB, Cassandra, or Firebase appear to be a natural fit for lenses. Tagging documents at write-time with a version would allow read-time translation of the data to any schema reachable via known lenses. Although our group did not attempt to implement lens support for any of these databases during this project, we believe most of our research findings and tools should be directly transferable to that environment and would love to hear from anyone interested in working on an implementation of these ideas for that environment.

SQL databases

There are plenty of industry blog posts describing the correct sequence of deploy steps required to rename a column in a database. Despite the fact that modern SQL databases like PostgreSQL support Multi-Version Concurrency for data, support for connecting to multiple schemas is limited.

For example, PostgreSQL allows most schema changes to occur within a transaction. Changing the schema for a table generally requires "locking" the table, preventing any reads or writes until the migration is complete. Worse still, after the migration is complete, the old schema is no longer available. Clients running code that has not been updated may now find their queries are no longer valid.

Lenses could even be integrated into PostgreSQL's query executor, translating written rows at read-time as required. This approach could enable support for multiple schemas, and reduce the cost of migrations. Updating the on-disk format of the data would be a performance optimization and run incrementally over time.

PostgreSQL does support updatable views, which in combination with the ability for clients to specify a "schema" at connection time could provide the basis for a user-land implementation of schemas and lenses. We are not aware of anyone who has tried to implement such a scheme.

Service-to-service synchronization

Instead of forcing each team or user to adopt the same software, what if we could write lenses that synchronize different web applications? We see an opportunity to connect different cloud services using a bidirectional lens that subscribes to each product's webhooks. Developers might file issues in a GitHub repository and their product manager might see the changes appear on a Trello board, while their contracted designer might track those same issues with other unrelated work in Asana. A set of lenses reading and writing webhooks—and customized to that team's workflows—could integrate all the services capably.

There are a number of interesting challenges in this approach. Not only do Asana and Trello model their data differently, but their data structures allow different kinds of manipulations. We could map a GitHub issue's Milestone to a Trello column, but what should we do if a new Trello column is added? How could we handle translating user accounts between the different services?

These problems may or may not be insurmountable, but if we could find a solution despite constraints, we have the potential to radically change the way people collaborate.

Towards a Cambrian era for software

Life on Earth first developed hard structures during the Cambrian period. Our local-first work with decentralized data was desperately in need of more structure, and that was the original inspiration for the project and its name. Before Cambria, we spent a lot of frustrating time trying to support even small differences in data structures between differing versions of our local-first software.

The Cambrian period is also known for being an era of rapid diversification of life on Earth. We believe that we are desperately in need of the same phenomenon in software right now.

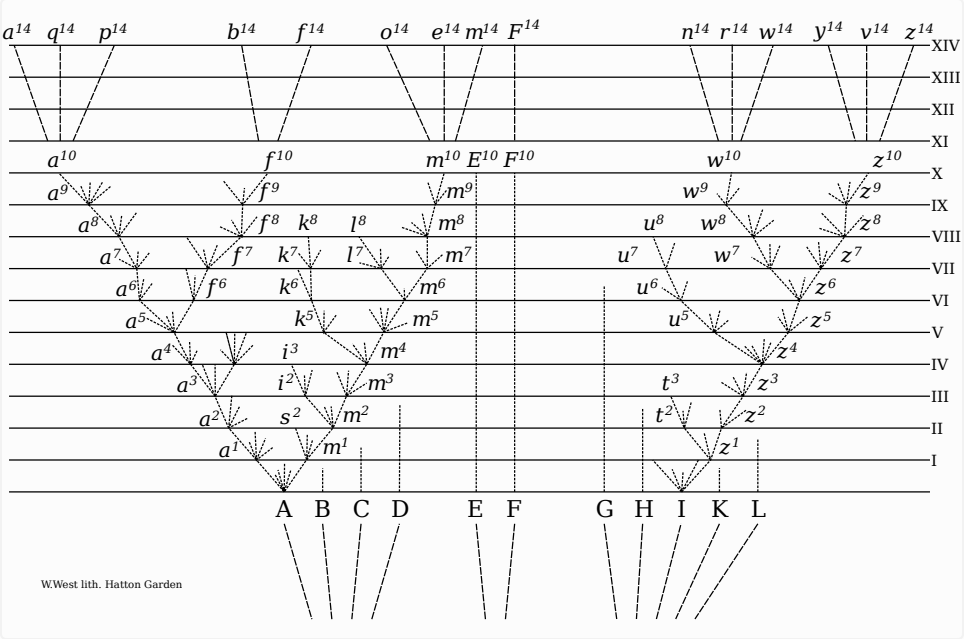
Today, much of the software we use is monolithic. It hoards data and is unavailable for user modification. We would like to end this. We believe software should be more diverse and better adapted to the people who use it.

The Convivial Computing Salon introduces itself by announcing that "we were promised bicycles for the mind, but we got aircraft carriers instead." Their annual salon is a venue for discussing software at a more human scale.

We have permitted our software to take ownership over our data. The migration of data from one application to another is often poorly supported or impossible, and real-time collaboration between programs is unheard of. We accept that to collaborate, we must

standardize on a program like GitHub Issues or Trello. We don't question why we can't open notes written in Evernote in Apple Notes, or vice versa.

A powerful lens-based system for collaboration could be precisely the lever required to connect these existing systems. Even better would be a world where software was built around a Cambria-like system from the beginning.



Darwin's Tree of Life is the only figure found in the Origin of Species.

As with the evolution of organic life, Cambrian software would be free to evolve in different directions without sacrificing compatibility. Programs could use lenses to maintain collaboration as they grow. Users and teams could adopt software that suits their needs, rather than standardizing on the lowest common denominator. Live collaboration could occur across diverging programs, with each team adding their own workflow features to an issue tracker while still able to collaborate on their shared work.

This vision may sound fanciful, but it is more than just a pipe dream. Today programmers already enjoy the benefit of being able to adopt editors as different as Visual Studio Code or Emacs, each customized to their exact preferences. We believe everyone should have the same flexibility.

Bringing about this change will not happen overnight, but you can start improving your own situation today. Every distributed system we have explored struggles with this problem. Just as programmers have embraced abstractions such as function calls and REST APIs, so too we hope that manual migration of data will become a thing of the past. If you're ready to get involved, consider contributing to Cambria or applying its concepts in your own domain.

We hope that you have enjoyed reading about these ideas, and if you are inspired to explore them further, or if you have questions we did not answer in this piece, we would love to hear from you: @inkandswitch or hello@inkandswitch.com.

One more thing: We already noted that all the examples in this project are live, running code, but if you press Ctrl-E, you can even edit them and see your changes live. It's a little rough around the edges, but we thought you might enjoy experimenting with them.

Thanks to everyone who offered feedback that improved this piece, including: Peter Alvaro, Herb Caudill, Scott Clasen, Blaine Cook, Christopher Dare, Jonathan Edwards, Carson Farmer, Dimitri Fontaine, Idan Gazit, Seph Gentle, Glenn Gillen, Daniel Jackson, Darius Kazemi, Martin Kleppmann, Clemens Nylandsted Klokmose, Brandur Leach, James Lindenbaum, Matt Manning, Todd Matthews, Mark McGranaghan, Trevor Morrison, Yoshiki Ohshima, Jeff Peterson, Jeremy Rose, Maciek Sakrejda, AJ Solimine, Matt Tognetti, Michael Toomim, and Adam Wiggins.

Appendices

Appendix I: Cambria implementation

In this section we briefly describe the implementation of the Cambria library, focusing on three key points:

- **Evolving patches:** For runtime data evolution, Cambria operates on *patches* to documents, not documents themselves.
- **Evolving schemas:** In addition to the runtime data context, lenses can also execute in a static schema context to produce updated schema definitions.
- **Bidirectionality:** A single lens specification translates data in both directions.

Evolving patches

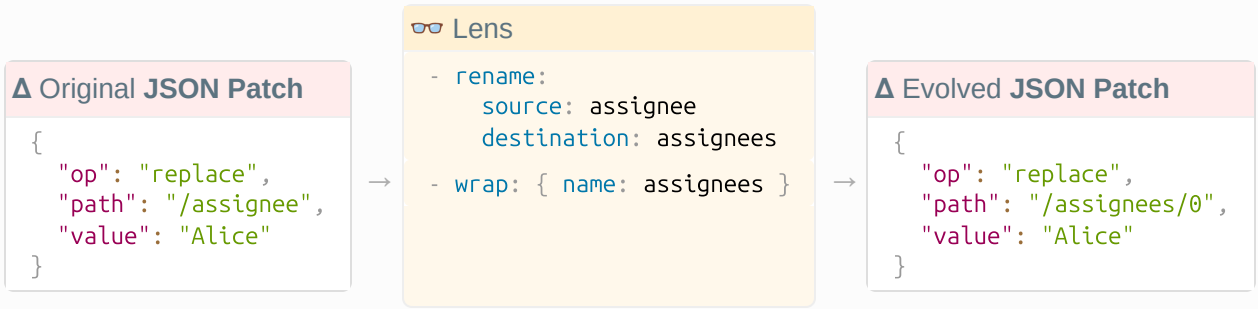
So far, it may appear that Cambria's focus is changing entire JSON documents from one format into another. But in fact, this isn't quite true. Under the hood, Cambria actually works with *patches* that represent modifications to a JSON document, evolving those patches between formats.

> This approach was directly inspired by work on "Edit Lenses", by Hofmann, Pierce, and Wagner.

Working with patches integrates smoothly with real-world systems, which often exchange patches between clients rather than copies of an entire data structure. Of course, it's still possible to convert a document with a patch that contains the entire contents of the document.

Concretely, Cambria lenses operate on patches in the JSON Patch standard, which describes changes to a JSON document in the form of a JSON document.

Below, we see a lens we encountered earlier, in which a scalar `assignee` field is mapped to an `assignees` array. But this time, instead of a document on each side, we see a patch representing an edit to a document. The original patch on the left sets the value of the `assignee` property. In the evolved patch on the right, the path has been modified to `/assignees/0`, which results in a patch that sets the first element of the `assignees` array.



Lenses transform JSON patches, not documents.

Evolving schemas

Cambria works in both runtime and build time contexts. In addition to the code that modifies patches, we have also built a separate lens interpreter that produces data schemas as output. This interpreter operates on JSON Schemas. Below, we once again see the same example lens, but this time operating on a JSON Schema description (which is also represented as JSON). The example lens both changes a property name and converts it from an optional scalar value to an array.

A lens transforms a JSON Schema representing the structure of a task.

Bidirectional operations

In many data evolution scenarios, it's important to translate data in two directions, back and forth. Without specialized tooling, this generally requires writing a pair of functions and hoping that they usefully correspond to each other in some way. The idea of bidirectional transformations is to write a single specification that can translate data in both directions and guarantee some useful properties.

Our work builds on lenses, which are a kind of bidirectional transformation first introduced by Foster, Greenwald, Kirkegaard, Pierce, and Schmitt. We specifically draw on the idea of "Edit Lenses" by Hofmann, Pierce, and Wagner—lenses that operate on *edits* to data structures, rather than the data structures themselves.

The definition of a valid edit lens is fairly intuitive. Given two documents and a lens between them, two properties must hold:

- When a valid edit happens on one document, and it's converted through the lens, it should become a valid edit on the other document. Applying the converted edit should never crash with a schema violation.
- There must be some *consistency relation* between the documents that still holds after edits are applied on both sides. This is the key to ensuring that the documents stay "synchronized" in some meaningful way.

Most of Cambria's operators fit this definition in a straightforward way. The simplest example is renaming a property. When a property is renamed from `name` to `title`, the consistency relation is that the value of `name` in the old document and `title` in the new must always be equal. It's easy to see how edits should be translated back and forth to

preserve this relation: Anytime the value of the `name` property is updated, the `title` property should also be updated with the same value, and vice versa.

The current implementation of Cambria does contain some operators that don't fit the technical definition of a lens. One example is the `convert` operator, which allows a developer to define separate forward and backward mappings between two data values. (This operator was used earlier, in the lens for converting from a boolean to a string value.) Because a developer could define any mapping they want in each direction, `convert` can't guarantee a useful consistency relation that will always hold between the two sides.

Nevertheless, this operator is still useful in practice; in our example lens, we were able to define a simple data mapping that was straightforward to reason about. We hypothesize that it's useful to sometimes allow users to manually specify each direction of a translation in cases where there is no convenient bidirectional mechanism, even if it does weaken the guarantees that can be provided about the translation logic.

Appendix II: Cambria-Automerge implementation

The core Cambria library only handles converting data patches and schemas, without any consideration for how a persistent document is stored, or how conflicts are resolved between collaborators. To add these components for our issue tracker, we integrated Cambria with the Automerge CRDT library and the Hypermerge persistence layer.

The main data structure in Automerge is a log of all changes that have been made to a document. Automerge guarantees a special property over this log: that any two users who have the same operations in the log will see the same document state, *regardless of the order of the operations.* This is a useful property for handling peer-to-peer editing.

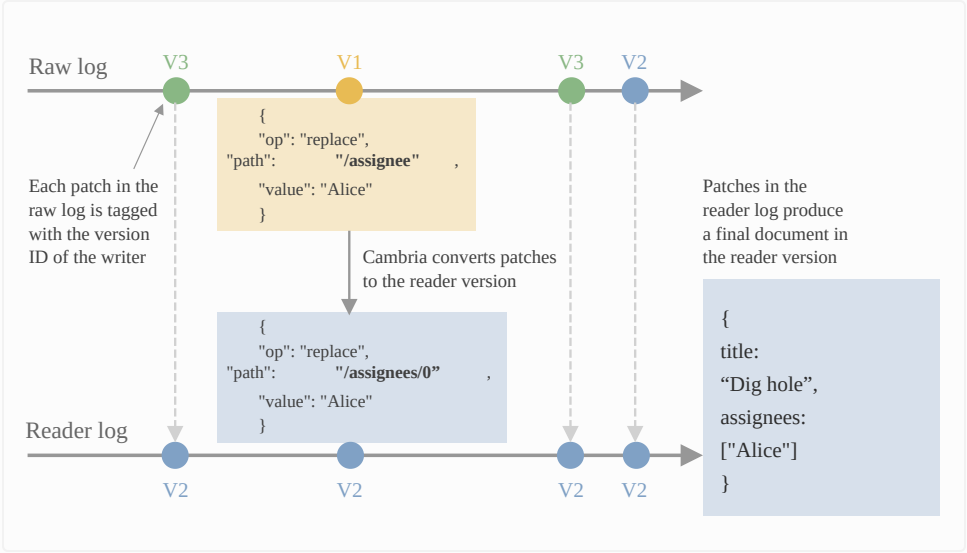
In Cambria-Automerge, we maintain this idea of an operation log, but with several extensions:

Schema-tagged writes

When a client makes a change, the new operation in the log is tagged with the writer schema—the schema that was used to make the change. To do this, we record a wrapper object in the Automerge log that contains a schema field in addition to the raw change.

Applying lenses on read

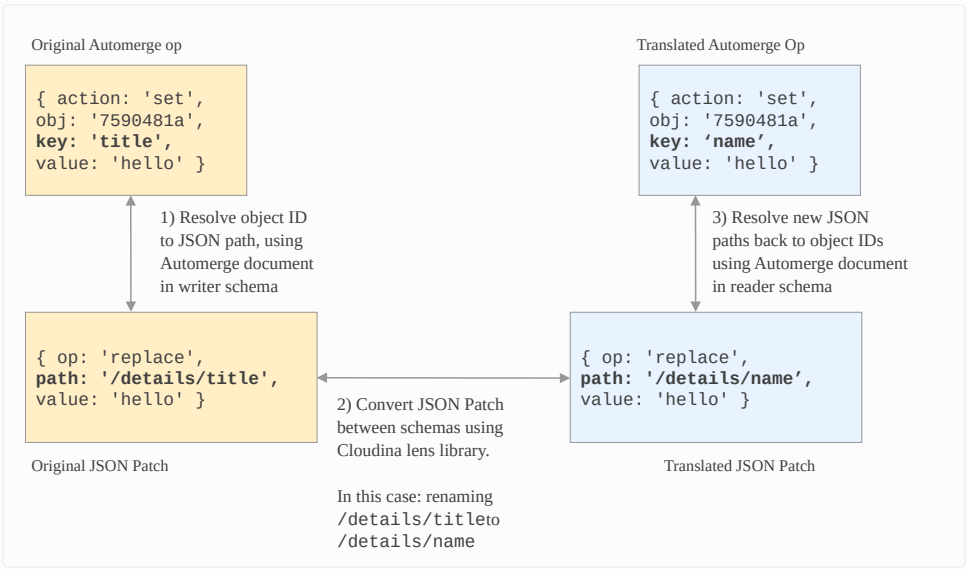
When a client applies the log of changes to read a document, each change is first evolved into the desired schema before being applied. Because the writer schema was saved, we can use Cambria to convert the patch from the writer schema to the reader schema before applying it. The result is a log of patches in terms of the reader schema, which can be replayed into a final document in that schema.



AUTOMERGE INTEGRATION

Cambria integrates with Automerge to produce a log of patches in the reader schema, regardless of the schemas used to write the patches.

One complication in the patch translation process is that Cambria uses the JSON Patch format to represent patches, whereas Automerge uses a different format better suited to conflict resolution. So, to apply a Cambria lens to a given Automerge patch (an “op”, in Automerge terminology), we need to do a three-step process: evolve to JSON Patch, use Cambria to lens the JSON Patch between two schemas, and then switch back to the Automerge patch format.



Translating an Automerge op requires converting it back and forth to a JSON Patch, which gets passed through Cambria.

A further complication is that Automerge operations refer to objects with IDs, but JSON Patches always use path names. Resolving back and forth between IDs and paths requires looking inside an Automerge document. More specifically, to perform this resolution process for a patch produced by some writer schema, we need to *read* the current state of the document in the writer schema at the time of the patch. As a result, reading a document in one schema can actually require reading it in many different schemas if writes were made from many schemas. This could be a source of performance problems, and a better solution might require a deeper integration between Automerge and Cambria.

Storing lenses in the document

A final extension is that lenses are stored in the document itself. When a client is writing to a document in a new schema that hasn't been used before in that document, it writes a special change to the log that contains the lens source as a JSON document.

This is critical so that collaborators can share lenses—if Alice starts writing to the document through a new lens, Bob needs a way to retrieve that lens and use it to translate Alice's changes into a format his client understands.

Summary

This design of the Cambria-Automerge integration has several useful properties:

- **Multi-schema**: The document can be read and written in any schema at any time. A document has no single "canonical" schema—just a log of writes from many schemas.
- **Self-evolving**: The logic for evolving formats is contained within the document itself. An older client can read newer versions without upgrading its code, since it can retrieve new lenses from the document itself.
- **Future-proof**: Edits to the document are resilient to the addition of future lenses. Since no evolution is done on write, old changes can be evolved using lenses that didn't even exist at the time of the original change.

One area for future research is further exploring the interaction between the guarantees provided by lenses and CRDTs. For example, Automerge has different logic for handling concurrent edits on a scalar value and an array. But what happens if the concurrent edits are applied to an array in one schema, and a scalar value in another?

Appendix III: On scalar-array conversions

When converting a field's type between an optional scalar (null or a value) and an array (zero or more values), we have to choose how the two sides will relate. We have already discussed in the Findings how there is no ideal solution, and in this Appendix we consider in detail the behaviour of a variety of different possible implementations.

Throughout this section we consider the *assignees* field for an issue tracker, and we can consider the value visible in the old version (denoted `{}` in examples) to be a window onto the head of the new version's array (denoted `[]`).

Reads

In the simplest case, both values are empty.

```
[{}]
```

With a single value, the read semantics are equivalent.

```
[{"Alice"}]
```

When further values are added to the new version, the old version would see just the head of the list.

```
[{ Alice }, Bob]
```

Writing

Writes originating from the new version are also straightforward, manipulating the array as they wish and leaving the old version to read whatever the first value is.

How should writes from the old version be interpreted by newer versions?

In these examples we'll look at what happens when the old version writes either "Eve" or `null`.

A defective implementation

Cambria currently implements a `head`/`wrap` lens pair that becomes inconsistent when the single-value side removes its value.

```
[ { Alice }, Bob, Charlie ] -- "Eve" -> [ {Eve}, Bob, Charlie ]
[ { Alice }, Bob, Charlie ] -- `null` -> {}, [Bob, Charlie ]
```

In this implementation, the view of the two sides is inconsistent. This is bad, and it leads to worse problems over time. Consider the next write:

```
{}, [ Bob, Charlie ] -- "Eve" -> [ {Eve}, Sue ]
{}, [ Bob, Charlie ] -- `null` -> {}, [ Charlie ]
```

Here, the single-value writer is now overwriting data they are unaware of.

Option 1: Old version rules

The simplest option is to give the old version control.

```
[ { Alice }, Bob ] -- "Eve" -> [ { Eve } ]
[ { Alice }, Bob ] -- `null` -> [{}]
```

The problem is that Eve didn't know "Bob" existed and has now clobbered him.

Option 2: Register

The old code only affects the 0th element, since that's all it can see.

```
[ { Alice }, Bob ] -- "Eve" -> [{Eve}, Bob]
[ { Alice }, Bob ] -- `null` -> [{null}, Bob]
```

This assumes the individual elements are nullable, which might be undesirable.

Option 3: Null clearing

The old code modifies just the element it sees on writes, but for deletes (set to null), it clears the whole array.

```
[ { Alice }, Bob ] -- "Eve" -> [{Eve}, Bob]
[ { Alice }, Bob ] -- `null` -> [{}]
```

The concern with this option is that the old code can delete data it isn't aware of. Eve might be surprised to learn she removed Bob from a project.

Option 4: Stack popping

Writes affect the head of the array, but writing a null is inferred to mean "delete the first element".

```
[ { Alice }, Bob ] -- "Eve" -> [{Eve}, Bob]
[ { Alice }, Bob ] -- `null` -> [{Bob}]
```

In this case, Eve might be surprised and confused that her "delete" has resulted in Bob appearing instead. The program has inferred that the intent of the delete was to remove just the head, and the old code can't know how many remaining values there are.

Option 5: Explicit articulation

Perhaps we should embrace the complexity of this space and insist that all scalars specify what kind of deletion they want in order to avoid getting into these problems in the future.

```
[ { Alice }, Bob ] -- "Eve" -> [{Eve}, Bob]
[ { Alice }, Bob ] -- `clear` -> [{}]
[ { Alice }, Bob ] -- `remove` -> [{Bob}]
```

This, of course, is a burden on developers in the present to spare potential problems in the future that would only manifest in rare occasions, but is perhaps useful to think about in terms of "intent preservation".

Option 6: Configurability

Instead of making a guess at the intent of the developer or the user, we could offer the option of customizing how to interpret null writes from older versions. They could:

- null the head element (requires nullable values in the array)
- clear the whole array
- remove the first element

On nullability

Notably, nullable data is annoying to deal with and requires frequent guards throughout developers' code to avoid creating runtime errors. Because of this, many fields are explicitly not nullable.

Not-nullable fields cannot be converted into arrays. This is because we can't guarantee a minimum (or maximum) array length in a distributed system.

On sets vs. arrays

In the case of issue assignees, we are using an array as a convenient or intuitive representation. In fact, because each assignee can only be on the task once, a set would be a better model. Automerge, our CRDT, doesn't have an explicit set type. We could emulate one with the keys of a dictionary, but that has its own challenges.